

ROMAN URDU STUDY NOTES

CSS Fundamentals Se Tailwind Tak

Box model, Flexbox, positioning, responsive design, Google Fonts, aur Tailwind CSS — sab kuch practical project (Chris Courses hero + stats section) ke sath explain kiya gaya hai.

Topics

Based On

Style

Display • Flexbox • Position • Media Queries • TailwindCSS Course Video TranscriptRoman Urdu + Code Examples

CSS Kya Hai?

CSS (Cascading Style Sheet) ek coding language hai jo HTML ko style karne ke liye use hoti hai — yani jo bhi HTML page banaya hai usko "pretty" banana. CSS hamesha HTML ke sath tandem mein use hoti hai — pehle HTML file honi chahiye, phir uske andar

`<style>` tag likh kar CSS shuru karte hain.

Class Banane Ka Tareeqa

CSS class ek **dot (.)** se start hoti hai, phir koi bhi naam, phir curly brackets `{ }` jinke andar properties likhi jati hain.

```
.text-red {  
  color: red; /* property : value ; */  
}
```

- **Property** — kya cheez style karni hai (jaise `color`)
- **Value** — kis tarah style karni hai (jaise `red`)
- Colon `:` property aur value ko separate karta hai
- Semicolon `;` statement complete hone ka signal deta hai

Class ko HTML element par apply karne ke liye `class="text-red"` attribute lagate hain. Ek element par multiple classes bhi laga sakte ho — koi limit nahi.

Tip: Ek CSS class mein jitni chahe properties daal sakte ho, aur poori site par jitni chahe classes bana sakte ho — mix-and-match karke exact look milta hai.

Project Ka Setup

Naya project shuru karne ke liye sirf ek file chahiye: `index.html` . Isko kisi bhi folder mein save karo (jaise `Web/css-course/`), phir double-click karke Chrome mein open karo. Har save ke baad refresh karke changes dekh sakte ho.

Figma Se Design Lena

Real-world client projects mein design pehle se **Figma** jaisi tool mein banaya hota hai. Developer ka kaam hota hai us design ko HTML/CSS mein replicate karna.

Kyun Figma?

Kisi bhi element par click karke uski exact properties (font family, size, weight, color) sidebar mein dikh jati hain — guessing ki zaroorat nahi.

Desktop + Mobile Design

Hamesha dono design maango — warna mobile site "garbage" lagegi. Developer hi responsible hota hai mobile responsiveness ke liye.

HTML Markup Likhne Ka Order

Design ko hamesha **top se bottom** markup karo:

1. **Header tag** — logo se sign-up button tak
2. **Div tags** — generic grouping ke liye (divider samajh lo)
3. **Anchor tags (<a>)** — links ke liye
4. **Button element** — CTA buttons ke liye

```
<header>
  <div>
    <img> <!-- logo -->
    <a>Pricing</a>
    <a>Courses</a>
  </div>
  <div>
    <a>Login</a>
    <button>Sign Up</button>
  </div>
</header>
```

Golden Rule: Sirf ek generic grouping ho to `<div>` use karo. Meaningful structure ke liye semantic tags (`header` , `section` , `nav`) use karo.

Background Color & Hexadecimal

background-color Property

```
.bg-gray-900 {  
  background-color: #12182b;  
}
```

Color 3 tareeqon se specify ho sakta hai: **named colors** (red, blue), **hexadecimal** (#ff0000), ya **rgba/hsl** values. Designers usually Figma se **hex code** dete hain.

Hexadecimal Format Samjho

Hex code 6 digits ka hota hai: **#RRGGBB** — har digit 0 se 9, ya a se f tak ho sakta hai.

Digits	Represent	Range
Pehle 2	Red	00 (dark) → ff (bright)
Beech ke 2	Green	00 (dark) → ff (bright)
Aakhri 2	Blue	00 (dark) → ff (bright)

```
background-color: #ff0000; /* pure red */  
background-color: #00ff00; /* pure green */  
background-color: #0000ff; /* pure blue */
```

Text Color

```
.text-white {  
  color: white;  
}
```

Dark background par default text (dark purple/black) legible nahi hota — **color: white** use karke contrast fix karte hain.

Design Tip: Har color ke liye ek "palette" banao (100 se 900 tak shades). Jitni zyada shades honge, utna better contrast aur focus control milega.

Browser Default Styling Fix Karna

Body Ka Default Margin

Browser by default `<body>` tag ko **8px margin** deta hai — isi wajah se page ke edges par unwanted spacing dikhti hai. Element ko **Inspect** karke (right-click → Inspect) is orange highlight (margin) ko dekh sakte ho.

```
body {  
  margin: 0; /* browser default margin hata do */  
}
```

Yaad rahe: Element ko select karke `.class` use karte hain, lekin body/html/head jaise auto-generated tags ko **directly tag name** se select karte hain — dot ki zaroorat nahi.

Inspect Element Ka Use

- **Orange border** = margin (element ke bahar ki spacing)
- **Green border** = padding (element ke andar ki spacing)
- Browser ke **Elements** tab mein live HTML/CSS dekh aur edit kar sakte ho (temporary, refresh par reset ho jata hai)
- **Computed** tab mein kisi bhi property ki default value dekh sakte ho

Pro Tip: Browser console mein directly CSS test karna fast hai, lekin permanent karne ke liye code copy karke apni HTML file mein paste karna zaroori hai.

Display Property

Display property decide karti hai ke element browser mein kaise layout hota hai. 5 main values hain:

Value	Behaviour
<code>block</code>	Parent container ki poori width le leta hai; naya element neeche push ho jata hai (div ka default)
<code>inline</code>	Sirf content jitni space leta hai; margin top/bottom kaam nahi karta (a tag ka default)
<code>inline-block</code>	Inline jaisa hi, magar margin/padding top-bottom bhi kaam karta hai
<code>none</code>	Element document flow se completely hat jata hai — invisible (popups, dropdowns ke liye)
<code>flex</code>	Children ko side-by-side layout karta hai + advanced alignment control deta hai

```
.block { display: block; }
.inline-block { display: inline-block; }
.inline { display: inline; }
.hidden { display: none; }
.flex { display: flex; }
```

Har HTML tag ka default display hota hai: `div` = block, `a` = inline. Computed tab mein check kar sakte ho.

Golden Rule: `display: flex` hamesha **parent** container par lagao — child elements par nahi. Flex parent apne saare children ko automatically affect karta hai, chahe wo pehle kuch bhi ho.

Flexbox Deep Dive

Flexbox professional development mein sabse zyada use hone wala layout tool hai. Sirf 3 classes se 90% kaam ho jata hai:

1. justify-content — Horizontal Alignment

```
.justify-between { justify-content: space-between; }  
.justify-center { justify-content: center; }
```

Children ko horizontal axis (left→right) par control karta hai. `space-between` pehla element far-left aur last element far-right rakhta hai, baaki equal spacing.

2. align-items — Vertical Alignment

```
.items-center { align-items: center; }
```

Children ko vertical axis par center karta hai (jaise logo aur nav links ko same line par align karna).

3. Nested Flex Containers

Flex classes ko multiple parent divs par "stack" kar sakte ho — jaise header ke andar do groups (logo+links, login+signup) ko phir header par bhi flex laga kar side-by-side kar sakte ho.

```
<header class="flex justify-between items-center">  
  <div class="flex items-center"> // logo + nav links  
  <div class="flex items-center"> // login + signup  
</header>
```

Reference: [css-tricks.com](https://css-tricks.com/a-complete-guide-to-flexbox/) ka "A Complete Guide to Flexbox" — sab justify-content aur align-items options ka visual guide milta hai (Flex-start, Flex-end, Center, Stretch, waghera).

Margin vs Padding

Margin

Element ke **bahar** ki spacing. Ek element ko doosre element se door karne ke liye use hota hai.

Padding

Element ke **andar** ki spacing (content aur background ke beech). Background color ke sath spacing chahiye ho to padding use karo.

Decision Rule: Agar background color hai aur content se gap chahiye → **padding**. Agar do alag elements (jaise 2 headers) ke beech gap chahiye → **margin**.

Individual Side Properties

```
padding-top: 18px;
padding-bottom: 18px;
padding-left: 12px;
padding-right: 12px;
```

Shorthand Syntax

Same syntax margin ke liye bhi kaam karta hai:

```
padding: 18px; /* sab sides same */
padding: 18px 12px; /* top-bottom 18px, left-right 12px */
padding: 18px 12px 100px 80px; /* top right bottom left (clockwise) */
```

Utility Classes (Atomic Approach)

```
.py-8 { padding-top: 8px; padding-bottom: 8px; }
.px-16 { padding-left: 16px; padding-right: 16px; }
```

CSS Cascade Warning: Jo class CSS mein **neeche** likhi ho wo upar wali ko override kar deti hai — chahe HTML mein order kuch bhi ho.

`.px-16` ke andar agar galti se `padding: 0 16px` likha (yani top/bottom bhi zero force kiya) to wo `.py-8` ko override kar dega. Isliye sirf specific properties set karo (`padding-left/right`), poora shorthand nahi.

Inline Styles — Bad Nahi Hain!

Ek unique, one-time style ke liye `style="padding: 18px 12px"` use karna theek hai — separation of concerns ka "old school" rule modern development mein utna zaroori nahi jitna pehle tha. Reusable styles ke liye class banao, unique styles ke liye inline theek hai.

Logo, Images & SVG

SVG vs PNG/JPEG

Format	Kab Use Karein
SVG	Icons, logos, graphics — vectorized, kabhi bhi pixelate nahi hota, size badhao to bhi crystal clear rehta hai
PNG / JPEG	Photos, complex images jinme bohat colors ho

```

```

Figma se logo lene ka tareeqa: element select karo → right-click → Copy as → Copy as SVG. Isko seedha HTML mein paste **na karo** (code messy ho jata hai) — nayi `.svg` file banao aur usko reference karo.

Relative Paths: `./img/logo.svg` ka matlab hai "isi directory ke andar img folder mein logo.svg". Files ko organize rakhne ke liye images ko alag `img/` folder mein rakho.

Width Control

```
.w-full { width: 100%; } /* parent container ki full width */
```

Image parent container se bahar overflow na ho, is liye width ko 100% set karna zaroori hai — warna page left-right scroll karne lagta hai.

Google Fonts Integrate Karna

Steps

1. **fonts.google.com** par jao, chahe wala font search karo (jaise "Lato")
2. Font style select karo (jaise Regular 400, Bold 700) — "Select this style"
3. Link tags copy karke `<head>` ke andar paste karo
4. CSS "specify families" rule copy karo aur apni class mein use karo

```
<!-- head ke andar -->
<link href="https://fonts.googleapis.com/..." rel="stylesheet">

/* CSS mein */
body {
  font-family: "Lato", sans-serif; /* fallback bhi rakho */
}
```

Best Practice: Font-family ko `body` tag par set karo, taake har child element automatically wahi font le — har element par alag se class lagane ki zaroorat nahi.

Multiple Weights (Regular + Bold)

Agar sirf "Regular 400" pull kiya aur `font-weight: bold` use kiya, to jin users ke computer par wo font install nahi hai unko bold nahi dikhega. Isliye Google Fonts se **donor weights** (400 aur 700) select karo:

```
<link href="...family=Lato:wght@400;700" >
```

Font Anti-Aliasing (Crisp Text)

Browsers by default text ko smooth nahi karte. Ye 2 lines hamesha `body` par add karo:

```
body {
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
```

Text Decoration, Transform & Weight

Underline Hatana

```
.no-underline { text-decoration: none; }
```

Anchor tags (`<a>`) by default underlined hote hain. Header/nav links mein underline zaroori nahi (user ko already pata hai ye clickable hai).

Uppercase Text

```
.uppercase { text-transform: uppercase; }
```

CTA buttons (jaise "Sign Up") ko attention grab karne ke liye uppercase kiya jata hai.

Font Weight

```
.font-bold { font-weight: bold; } /* ya 700 */
```

Weight scale 100 (thin) se 900 (very bold) tak hota hai. Values: `bold` , `bolder` , `lighter` , `normal` , ya numeric.

Font Size & Letter Spacing

```
.text-sm { font-size: 14px; }  
letter-spacing: 1px; /* characters ke beech spacing */
```

Default Font Family Trap: Buttons ka default font family `Arial` hota hai — chahe body mein koi bhi font family set ho! Fix karne ke liye `button` tag ko explicitly target karo:

```
button { font-family: "Lato", sans-serif; }
```

Border & Border Radius

Border Hatana

Buttons ko default mein ek ugly "beveled" border milta hai — modern design ke liye hata do:

```
.border-none { border: none; }
```

Rounded Corners

```
.rounded-full { border-radius: 9999px; } /* pura pill-shape */  
border-radius: 4px; /* halka sa rounding */
```

Jitna zyada `border-radius` value, utna zyada curve. Bohat high value (jaise 9999px) hamesha fully-rounded pill shape deta hai, chahe padding kuch bhi ho.

Complete Button Recipe

```
.btn-primary {  
  background-color: #5e9eff;  
  color: white;  
  border: none;  
  border-radius: 9999px;  
  padding: 12px 28px;  
  font-weight: bold;  
  text-transform: uppercase;  
  font-size: 14px;  
  letter-spacing: 1px;  
}
```

Clickable Area Tip: Links ke beech spacing ke liye **margin** ki jagah **padding** use karo — isse whitespace bhi clickable ban jata hai, user experience better hota hai.

Position: Relative & Absolute

position: relative

```
.relative { position: relative; }
```

Element apni original position se `top`, `right`, `bottom`, `left` properties se move ho sakta hai, lekin document flow mein space still reserve rehti hai.

position: absolute

```
.absolute { position: absolute; }
```

Element document flow se **completely nikal jata hai** — dusre elements is par depend nahi karte. Ye apne **nearest positioned parent** (jis ka position relative/absolute ho) ke relative position hota hai.

Critical Rule: Agar koi parent element mein `position: relative` nahi hai, to absolute child seedha **browser window** ke relative position ho jayega — jo galat layout create karta hai. Hamesha parent-child pair banao.

Z-Index — Layering

```
.z-10 { z-index: 10; }
```

Z-axis (screen ke andar-bahar) positioning — **bada number** = closer to viewer (foreground), **chhota number** = background. Z-index sirf tab kaam karta hai jab element ka position `relative` ya `absolute` ho.

inset-0 Shortcut

```
.inset-0 {  
  top: 0; right: 0;  
  bottom: 0; left: 0;  
}
```

Ye 4 properties ek sath zero par set karke element ko parent ki **poori width/height** tak stretch kar deta hai — overlays banane ke liye perfect (jaise dark overlay video ke upar).

Background Video & Object-Fit

Video Tag Setup

```
<video
  loop autoplay playsinline muted
  src="./videos/galaxy.mp4"
></video>
```

Attribute	Kaam
loop	End hone par phir se shuru se play
autoplay	User click ke bagair khud play ho
playsinline	Mobile par inline play (fullscreen force nahi karta)
muted	Zaroori — bina mute autoplay kaafi browsers mein block ho jata hai

Absolute vs Relative URL: Kisi doosri site ka video seedha use na karo (absolute URL) — agar wo site apna video change/delete kare to tumhara site break ho jayega. Video download karke apni `videos/` folder mein rakho aur relative path use karo.

object-fit: cover

```
.object-cover { object-fit: cover; }
```

Video/image ko parent container ke **exact height aur width** tak fill karta hai — chahe screen size kuch bhi ho — aur **aspect ratio maintain** karta hai (zoom karke, stretch nahi karke). Bina isko blank spacing dikhti hai jab screen resize hoti hai.

Containers & Max-Width

Bade screens (jaise 4K monitor) par agar content full-width le le, to text bohat dur left/right chala jata hai jo hard-to-read banata hai. Solution: **container pattern**.

Max-Width

```
max-width: 1200px; /* is se zyada width nahi le sakta */
```

`max-width` aur set `width` mein farq: `max-width` chhoti screens par scale down ho sakta hai, lekin large screens par ek limit se zyada nahi grow karega.

Auto-Centering

```
.mx-auto {  
  margin-left: auto;  
  margin-right: auto;  
}
```

`margin: auto` browser ko automatically equal margin left aur right calculate karne deta hai — element perfectly center ho jata hai.

```
<div class="container mx-auto" >  
  <!-- max-width: 1200px + auto centering -->  
</div>
```

Important: Container class ko un elements par mat lagao jinhe full-width chahiye (jaise background video) — sirf text/content wrapper divs par lagao, warna video bhi 1200px tak limit ho jayegi.

Media Queries & Viewport

Viewport Meta Tag — Sabse Zaroori

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" >
```

Ye tag har project mein hamesha daalo! Bina is ke, Chrome DevTools ka mobile emulator (aur real phones) sahi tarah scale nahi karte — sab kuch galat dimensions mein render hoga.

Media Query Syntax

```
/* Sab kuch bahar = mobile styling (default) */

@media (min-width: 640px) {
  /* Ye sab sirf 640px se badi screens par chalega */
  .bg-blue { background-color: blue; }
}
```

Common Breakpoints

Naam	Min-Width	Device
sm	640px	Large phones
md	768px	Tablets
lg	1024px	Small desktops
xl	1280px	Desktops
2xl	1536px	Large desktops

Mobile-First Approach

Sab default styles **mobile ke liye** likho, aur media query ke andar sirf wo cheezein overwrite karo jo desktop par different chahiye (jaise bada font-size, extra padding, navigation dikhana).

Emulator Tip: Chrome DevTools mein "Toggle Device Toolbar" icon se different screen sizes emulate kar sakte ho. Agar viewport tag add karne ke baad emulator sahi nahi dikh raha, to emulator ko band karke dobara open karo.

Hide/Show Elements Par Screen Size

Chhoti screens par bohat sare nav links fit nahi hote — solution hai unhe hide karna aur hamburger icon dikhana.

```
/* Mobile: nav hidden, hamburger visible */
nav { display: none; }
.hamburger { display: block; }

/* Tablet+ (768px se upar): nav visible, hamburger hidden */
@media (min-width: 768px) {
  nav { display: block; }
  .hamburger { display: none; }
}
```

SVG Icons — Hero Icons

heroicons.com se free SVG icons directly copy-paste kar sakte ho. SVG ka color set karne ke liye:

```
.hamburger-icon { color: white; } /* stroke:currentColor use karta hai */
```

SVG mein `stroke="currentColor"` ho to wo automatically parent ke `color` property se apna color leta hai.

Flex + Hidden Interaction: Agar koi element `display: none` se hidden hai, to Flexbox use bilkul ignore kar deta hai jaise wo exist hi nahi karta — space allocation mein include nahi hota.

Tailwind CSS — Introduction

Tailwind CSS ek utility-first CSS framework hai jo pre-built classes deta hai (jaise `flex` , `pt-4` , `text-center`) — jo basically wahi classes hain jo hum manually likh rahe the, lekin ready-made.

CDN Se Install Karna (Beginner-Friendly)

```
<!-- head ke andar, jitni jaldi ho sake -->
<script src="https://cdn.tailwindcss.com"></script>
```

Production ke liye recommended nahi (Tailwind khud batata hai), lekin seekhne aur prototyping ke liye sabse easy tareeqa hai.

Browser Reset Ka Asar

Tailwind import karte hi ek **reset stylesheet** automatically apply hoti hai — sab HTML elements ki default styling (jaise H1 ka bold, ya default line-height) zero ho jati hai.

- H1 ab bold nahi rahega → manually `font-bold` lagana padega
- Line-height change ho sakti hai → text ka spacing tight ho sakta hai
- Apne banaye hue duplicate classes hata do (jaise apna khud ka `.flex` , `.hidden`) — Tailwind pehle se deta hai

Line-Height Fix

```
.leading-none { line-height: 1; } /* font-size ke barabar */
.leading-tight { line-height: 1.25; }
.leading-loose { line-height: 2; }
```

Search Karna Seekho: tailwindcss.com par direct search bar se koi bhi CSS property search karo (jaise "font-bold" ya "opacity") — Tailwind exact class name dikha deta hai. Apni khud ki class likhne se pehle hamesha check karo Tailwind mein already available to nahi.

Tailwind Responsive Prefixes

Tailwind manually media-query likhne ki zaroorat khatam kar deta hai — bas class name ke aage breakpoint prefix laga do.

Prefix	Min-Width
sm:	640px
md:	768px
lg:	1024px
xl:	1280px
2xl:	1536px

Escape Character

Colon `:` CSS mein special meaning rakhta hai, isliye jab manually class banate ho to backslash se "escape" karna padta hai:

```
.sm\:text-60px { font-size: 60px; }  
/* HTML mein use: class="sm:text-60px" (backslash nahi likhte) */
```

Chaining Breakpoints

```
<a class="hidden sm:inline lg:hidden">Pricing</a>
```

Ye example: mobile par hidden → 640px se dikhna shuru → 1024px par phir hidden ho jaye (jab bade screen par bhi hamburger chahiye ho).

Practical Example — Nav Hide/Show

```
<nav class="hidden md:block"> ... </nav>  
<svg class="md:hidden text-white"> ... </svg> <!-- hamburger -->
```

EM vs REM Units

em unit

Parent element ke font-size ke relative. Agar parent 16px hai aur child `1.25em` hai, to child = 20px. Parent change ho to child bhi change hoga (nested confusion ho sakta hai).

rem unit

Sirf root `<html>` tag ke font-size ke relative (default 16px).
Consistent aur predictable — **zyada recommended**.

```
/* html { font-size: 16px } default hota hai */  
font-size: 1.25rem; /* = 20px */  
margin-top: 1rem; /* = 16px */
```

Tailwind Ka Spacing Scale

Tailwind apna standardized rem-based scale deta hai — font-size, margin, padding sab isi scale se follow karte hain:

Class Suffix	REM	Pixels
1	0.25rem	4px
2	0.5rem	8px
4	1rem	16px
8	2rem	32px
16	4rem	64px
32	8rem	128px

```
<div class="mt-4 px-6 pt-32"> ... </div>
```

Fayda: Ek consistent scale use karne se pura site "harmonious" lagta hai — random 17px, 23px jaisi values se bachte hain.

Opacity, Overlays & Gradients

Opacity

```
.opacity-50 { opacity: 0.5; } /* 0 = invisible, 1 = fully visible */
```

Background video par text legible banane ke liye ek **black overlay div** (position: absolute + inset-0) daalte hain jiski opacity 50% hoti hai.

Linear Gradient

```
.bg-gradient-overlay {  
  background-image: linear-gradient(to bottom, black, transparent);  
}
```

Solid overlay ki jagah gradient zyada natural lagta hai — top par dark (black), neeche fade hoke transparent. Text jahan hai wahan darkness zyada, baaki jagah kam.

```
<div class="absolute inset-0 bg-gradient-to-b from-black to-transparent"></div>
```

Overlay Ka Height Control

Agar gradient/overlay ko default se zyada tak fade karna hai, to `inset-0` ke bottom value ko negative karke div ko taller bana do (jaise `bottom: -35px` inline style se override).

Responsive Consideration: Overlay/gradient sirf desktop par jaruri ho to `hidden md:block` use karo — mobile par simpler black overlay hi kaafi hai.

Divide Utility & Width Fractions

Equal-Width Flex Children

```
.w-1\3 { width: 33.333%; }
```

3 stat boxes (jaise "500+ Lessons", "100+ Hours", "5000+ Students") ko equal width dene ke liye percentage-based width use karo — static pixels nahi, taake responsive resize par sab proportion maintain rahe.

Divide-X — Automatic Dividers

```
<div class="flex divide-x divide-gray-600">  
  <div class="w-1/3">Box 1</div>  
  <div class="w-1/3">Box 2</div>  
  <div class="w-1/3">Box 3</div>  
</div>
```

`divide-x` automatically har child ke **beech** mein ek vertical border laga deta hai (edges par nahi) — manually `border-left/right` lagane se better tareeqa.

Manual Borders (Alternative)

```
.border-b { border-bottom-width: 1px; }  
.border-solid { border-style: solid; }  
.border-gray-500 { border-color: #6b7280; }
```

3 Cheezein Zaroori: Border dikhne ke liye width + style + color teeno chahiye hote hain — sirf color set karne se kuch nahi hoga.

Quick Cheat Sheet

Category	Classes
Display	block, inline, inline-block, hidden, flex
Flex	justify-between, justify-center, items-center
Spacing	p-*, px-*, py-*, m-*, mx-auto, mt-*
Position	relative, absolute, inset-0, z-10 → z-50
Sizing	w-full, w-1/3, h-full, max-w-*
Typography	text-white, font-bold, uppercase, no-underline, leading-none
Border	border-none, rounded-full, border-b, divide-x
Effects	opacity-50, bg-gradient-to-b, from-black, to-transparent
Responsive	sm:, md:, lg:, xl:, 2xl:

Poore Workflow Ka Summary

1. Figma design se properties nikaalo (font, color, spacing, width)
2. HTML markup top-to-bottom likho (semantic tags use karo)
3. Mobile-first CSS likho, phir media queries se desktop enhance karo
4. Container + max-width + mx-auto pattern se large screens control karo
5. Repetitive inline styles ko utility classes mein convert karo
6. Jab pattern clear ho jaye, Tailwind CSS import karke development speed badhao

Next Steps: Popular Courses section, Features, Testimonials slider, Pricing, FAQ, aur Footer — same principles (Flexbox + Position + Tailwind) se develop honge.